

High Performance Computing

CS5013

Lab - 09

Student Details

Name: R Abinav

Roll No: ME23B1004

Date: March 20, 2026

Source Code

Listing 1: parallel nxn matrix multiplication using cuda

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>

#define N 1024

__global__ void matrix_mul(double *a, double *b, double *c, int n)
{
    int row = blockDim.y * blockIdx.y + threadIdx.y;
    int col = blockDim.x * blockIdx.x + threadIdx.x;

    if(row < n && col < n)
    {
        double sum = 0.0;
        for(int k = 0; k < n; k++)
        {
            sum += a[row * n + k] * b[k * n + col];
        }
        c[row * n + col] = sum;
    }
}

void serial_matrix_mul(double **a, double **b, double **c, int n)
{
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
        {
            double sum = 0.0;
            for(int k = 0; k < n; k++)
                sum += a[i][k] * b[k][j];
            c[i][j] = sum;
        }
}
```

```

}

int main()
{
    int n = N;
    size_t bytes = (size_t)n * n * sizeof(double);

    printf("matrix_multiplication_of_%dx%d_matrices\n", n, n);
    printf("element_type: double_precision\n");
    printf("total_elements_per_matrix: %d\n\n", n * n);

    double **a = (double**)malloc(n * sizeof(double*));
    double **b = (double**)malloc(n * sizeof(double*));
    double **c_serial = (double**)malloc(n * sizeof(double*));
    for(int i = 0; i < n; i++)
    {
        a[i] = (double*)malloc(n * sizeof(double));
        b[i] = (double*)malloc(n * sizeof(double));
        c_serial[i] = (double*)malloc(n * sizeof(double));
    }

    srand(42);
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
        {
            a[i][j] = ((double)rand() / RAND_MAX) * 10.0;
            b[i][j] = ((double)rand() / RAND_MAX) * 10.0;
        }

    clock_t start_serial = clock();
    serial_matrix_mul(a, b, c_serial, n);
    clock_t end_serial = clock();
    double serial_time = ((double)(end_serial - start_serial)) /
        CLOCKS_PER_SEC;

    printf("serial_execution_completed\n");
    printf("serial_time: %f_seconds\n\n", serial_time);

    double *h_a = (double*)malloc(bytes);
    double *h_b = (double*)malloc(bytes);
    double *h_c = (double*)malloc(bytes);

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
        {
            h_a[i * n + j] = a[i][j];
            h_b[i * n + j] = b[i][j];

```

```

    }

    double *d_a, *d_b, *d_c;
    cudaMalloc(&d_a, bytes);
    cudaMalloc(&d_b, bytes);
    cudaMalloc(&d_c, bytes);

    cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, h_b, bytes, cudaMemcpyHostToDevice);

    dim3 threads(16, 16);
    dim3 blocks((int)ceil((float)n / threads.x), (int)ceil((float)n
        / threads.y));

    printf("configuration: \t blocks \t\t (%d, \t%d), \t threads \t\t (%d, \t%d)\n",
        blocks.x, blocks.y, threads.x, threads.y);

    cudaEvent_t start_p, stop_p;
    cudaEventCreate(&start_p);
    cudaEventCreate(&stop_p);
    cudaEventRecord(start_p);

    matrix_mul<<<blocks, threads>>>(d_a, d_b, d_c, n);

    cudaEventRecord(stop_p);
    cudaEventSynchronize(stop_p);
    float parallel_ms = 0;
    cudaEventElapsedTime(&parallel_ms, start_p, stop_p);
    double parallel_time = parallel_ms / 1000.0;

    cudaMemcpy(h_c, d_c, bytes, cudaMemcpyDeviceToHost);

    double max_diff = 0.0;
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
        {
            double diff = fabs(h_c[i * n + j] - c_serial[i][j]);
            if(diff > max_diff) max_diff = diff;
        }

    printf("\n");
    if(max_diff < 1e-3)
        printf("verification: \t success \t (max \t diff \t = \t %e)\n", max_diff);
    else
        printf("verification: \t failed \t (max \t diff \t = \t %e)\n", max_diff);

    printf("parallel \t time: \t %f \t seconds\n", parallel_time);

```

```

printf("speedup: %.2fx\n", serial_time / parallel_time);

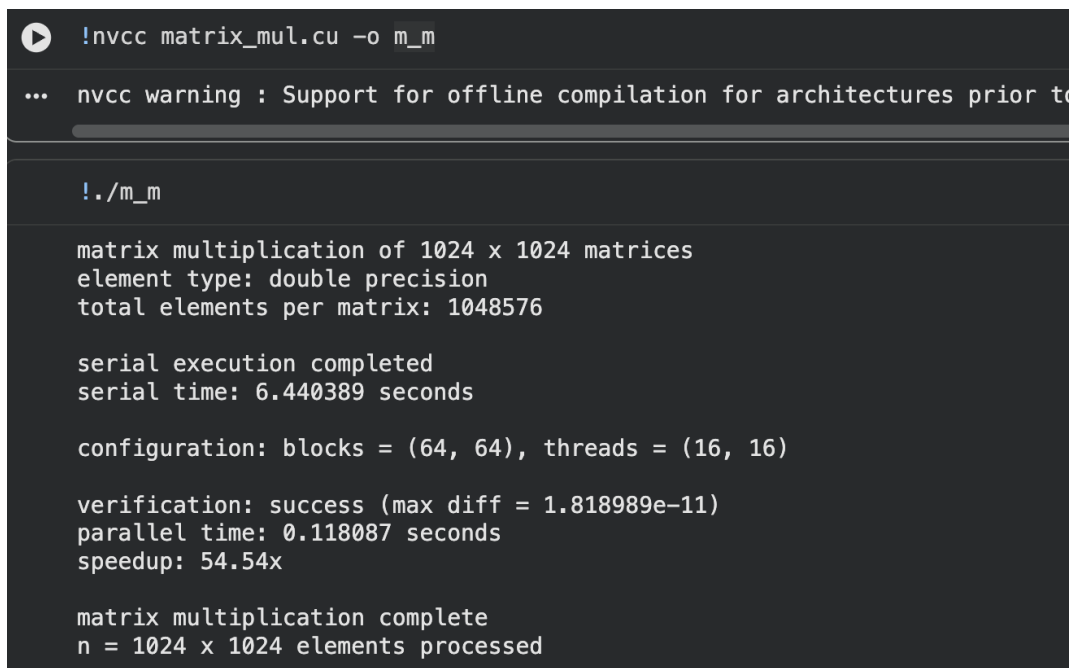
for(int i = 0; i < n; i++)
{
    free(a[i]); free(b[i]); free(c_serial[i]);
}
free(a); free(b); free(c_serial);
free(h_a); free(h_b); free(h_c);
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);

printf("\nmatrix multiplication complete\n");
printf("n= %d x %d elements processed\n", n, n);

return 0;
}

```

Output



```

!nvcc matrix_mul.cu -o m_m
... nvcc warning : Support for offline compilation for architectures prior to
...

!./m_m

matrix multiplication of 1024 x 1024 matrices
element type: double precision
total elements per matrix: 1048576

serial execution completed
serial time: 6.440389 seconds

configuration: blocks = (64, 64), threads = (16, 16)

verification: success (max diff = 1.818989e-11)
parallel time: 0.118087 seconds
speedup: 54.54x

matrix multiplication complete
n = 1024 x 1024 elements processed

```

Figure 1: output of cuda matrix multiplication showing serial time, parallel time, and speedup for 1024 x 1024 double precision matrices